Scheduling Algorithms

Why Scheduling?

Multiprogramming aims to maximize CPU utilization by switching the CPU among various processes, thus keeping the OS productive. Several processes reside in memory simultaneously, and when one process waits (e.g., for I/O), the OS assigns the CPU to another process. Scheduling is a fundamental OS function that enables this.

CPU I/O Burst Cycle

Process execution is characterized by a cycle of CPU execution and I/O wait. Processes alternate between these two states.

- A process begins with a CPU burst, followed by an I/O burst, and so on.
- The final CPU burst concludes with a system request to terminate execution.
- The distribution of CPU bursts is a primary consideration in scheduling.

Scheduling Algorithm Optimization Criteria

The criteria for optimizing scheduling algorithms include:

1. CPU Utilization: Keep the CPU as busy as possible, aiming for utilization between 40% (lightly loaded) to 90% (heavily loaded).
2. Throughput: Maximize the number of processes that complete their execution per time unit.
3. Turnaround Time: Minimize the time it takes to execute a particular process, from submission to completion.
4. Waiting Time: Minimize the amount of time a process spends waiting in the ready queue.
5. Response Time: Minimize the time it takes to produce the first response, not output, in a time-sharing environment.

These can be summarized as:

- Maximize CPU utilization
- Maximize throughput
- Minimize turnaround time
- Minimize waiting time
- Minimize response time

CPU Scheduler

The **short-term scheduler** selects a process from the **ready queue** and allocates the CPU to it. The ready queue can be ordered in various ways. CPU scheduling decisions occur when a process:

1. Switches from running to waiting state (e.g., I/O request).
2. Switches from running to ready state (e.g., interrupt or timer expiration).
3. Switches from waiting to ready state (e.g., completion of I/O).
4. Terminates.

Scheduling scenarios 1 and 4 are **non-preemptive**, while others are **preemptive**. Considerations include access to shared data, preemption in kernel mode, and interrupts during critical OS activities.

Non-Preemptive vs. Preemptive Scheduling

- **Non-Preemptive Scheduling**: Once the CPU is allocated to a process, it retains the CPU until it terminates or switches to the waiting state. An example is Microsoft Windows 3.x.
- **Preemptive Scheduling**: Introduced in Windows 95, preemptive scheduling is used in subsequent Windows operating systems.

Dispatcher

The **dispatcher** module grants control of the CPU to the process selected by the short-term scheduler. This involves:

- Switching context.
- Switching to user mode.
- Jumping to the correct location in the user program to restart it.

Dispatch latency is the time it takes for the dispatcher to stop one process and start another.

Non-Preemptive Algorithms

First-Come, First-Served (FCFS) Scheduling

> The simplest CPU scheduling algorithm where the process that requests the CPU first is allocated the CPU first.

FCFS is implemented using a FIFO queue. When a process enters the ready queue, its PCB is linked to the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue. The average waiting time for FCFS can be quite long.

Example

Consider three processes with the following burst times:

Process	Burst Time
P1	24
P2	3
P3	3

If the processes arrive in the order P1, P2, P3, the Gantt Chart is:

```
P1   P2   P3
0   24   27   30
```

Waiting time for P1 = 0; P2 = 24; P3 = 27. Average waiting time: (0 + 24 + 27) / 3 = 17.

If the processes arrive in the order P2, P3, P1, the Gantt Chart is:

```
P2   P3   P1
0   3   6   30
```

Waiting time for P1 = 6; P2 = 0; P3 = 3. Average waiting time: (6 + 0 + 3) / 3 = 3.

FCFS is non-preemptive, and once the CPU is allocated to a process, that process retains it until termination or I/O request. It is not suitable for time-sharing systems.

Convoy Effect

> The convoy effect is a phenomenon associated with FCFS where a short process is stuck behind a long process, causing the entire OS to slow down due to a few large processes tying up system resources.

Formulae

- Turn Around Time = Completion Time - Arrival Time
- Waiting Time = Turn Around Time - Burst Time or Sum of times spent waiting in Ready queue - Arrival Time
- Average waiting time = Total_waiting_time / no_of_processes
- Average turnaround time = Total_turn_around_time / no_of_processes

Shortest-Job-First (SJF) Scheduling

> An algorithm that selects the process with the smallest execution time for the next execution.

SJF associates with each process the length of its next CPU burst and uses these lengths to schedule the process with the shortest time. SJF is optimal, providing the minimum average waiting time for a given set of processes.

Two Schemes:

1. Non-Preemptive: Once a process starts, it runs to completion. SJF is optimal if all processes are ready simultaneously.
2. Preemptive: Also known as the Shortest-Remaining-Time-First (SRTF). SRTF is optimal if processes arrive at different times.

Example of Non-Preemptive SJF

Process	Arrival Time	Burst Time
P1	0.0	6
P2	2.0	8
P3	4.0	7
P4	5.0	3

SJF scheduling chart:

```
P4   P1   P3   P2
0   3   9   16   24
```

Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

Example for Non-Preemptive SJF with Different Arrival Times

Process	Arrival Time	Burst Time
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

SJF (non-preemptive)

```
P1   P3   P2   P4
0   7   8   12   16
```

At time 0, P1 is the only process, so it gets the CPU and runs to completion. After P1 completes, the queue holds P2, P3, and P4. P3 gets the CPU first since it is the shortest, then P2, then P4.

Process	Arrival Time	Burst Time	Waiting Time	Turnaround Time
P1	0.0	7	0	7
P2	2.0	4	6	10
P3	4.0	1	3	4
P4	5.0	4	7	11

Round Robin (RR)

> Designed for time-sharing systems, RR is similar to FCFS but adds preemption. A small unit of time, called a **time quantum** or **time slice**, is defined (typically 10-100 milliseconds).

The ready queue is treated as a **circular queue**. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

How RR Works

- The ready queue uses FIFO order.
- New processes are added to the tail of the ready queue.
- The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.

Two scenarios:

1. If a process has a CPU burst less than 1 TQ, it voluntarily releases the CPU.
2. If a process has a CPU burst greater than 1 TQ, the timer goes off, causing an interrupt to the OS. A context switch is performed, and the process is placed at the tail of the ready queue.

Example of RR with Time Quantum = 4

Process	Burst Time
P1	24
P2	3
P3	3

The Gantt chart is:

```
P1   P2   P3   P1   P1   P1   P1   P1
0   4   7   10   14   18   22   26   30
```

Waiting Time: P1 = (0 + (10-4)) = 6; P2 = 4; P3 = 7. Average Waiting Time (AWT) = (6+4+7)/3 = 17/3

RR is preemptive. If there are n processes in the ready queue and the time quantum is q, each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.

Performance

- If TQ is large, RR is the same as FIFO.
- If TQ is small, RR is called processor sharing, appearing to the user as if each of n processes has its own processor running at $1/n$ the speed of the real processor.
- q must be large with respect to context switch time; otherwise, overhead is too high.

Time Quantum and Context Switch Time

The TQ should be large relative to the context switch time. Typically, TQ is 10ms to 100ms, while a context switch is less than 10usec. Otherwise, much time is wasted in context switching.

Turnaround Time

Typically, RR has a higher average turnaround than SJF but better response. The average turnaround time of a set of processes does not necessarily improve as the time quantum increases. The ATT can be improved if most processes finish their next CPU burst in a single TQ.

Thumb Rule: 80% of CPU bursts should be shorter than TQ.

Pre-emptive Algorithms

Shortest Remaining Time First (SRTF)

In the Shortest Remaining Time First (SRTF) scheduling algorithm, the process with the smallest amount of time remaining until completion is selected to execute.

Example for Preemptive SJF (SRTF)

Process	Arrival Time	Burst Time
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

```
P1   P2   P3   P2   P4   P1
0    2    4    5    7    11   16
```

- Time 0: P1 gets the CPU. Ready = [(P1,7)]
- Time 2: P2 arrives. CPU has P1 with rem. time = 5. Ready = [(P2,4)]. P2 gets the CPU, P1 is preempted.
- Time 4: P3 arrives. CPU has P2 with rem. time = 2. Ready = [(P1,5),(P3,1)]. P3 gets the CPU, P2 is preempted.
- Time 5: P3 completes and P4 arrives. Ready = [(P1,5),(P2,2),(P4,4)]. P2 gets the CPU.
- Time 7: P2 completes. Ready = [(P1,5),(P4,4)]. P4 gets the CPU.
- Time 11: P4 completes, and P1 gets the CPU.

Waiting Time for Processes: P1 = (0 + (11-2)) = 9; P2 = ((2-2) + (5-4)) = 1; P3 = (4-4) = 0; P4 = (7-5) = 2. Average waiting time = (9 + 1 + 0 + 2) / 4 = 3.

Estimating the Length of Next CPU Burst

A problem with SJF is that it is difficult to know the length of the next CPU burst. One idea is to predict based on recent observations.

Exponential Averaging

The next CPU burst t_{n+1} is estimated using exponential averaging:

$t_{n+1} = \alpha t_n + (1 - \alpha) t_{n-1}$

Where:

- t_n = nth CPU burst (most recent information)
- t_{n-1} = average of all past bursts (past history)
- α = weighting factor between 0 and 1

α controls the relative weight of recent and past history in our prediction. If α is close to 1, recent history matters more, while if α is close to 0, past history is more relevant.

Examples of Exponential Averaging

- If $\alpha = 0$, then $t_{n+1} = t_{n-1}$ Recent history does not matter.
- If $\alpha = 1$, then $t_{n+1} = t_n$ Only the most recent CPU burst matters.

More commonly, $\alpha = 1/2$, so recent history and past history are equally weighted.

The initial t_0 can be defined as a constant or as an overall system average.

Priority Scheduling

> A priority number (integer) is associated with each process. The CPU is allocated to the process with the highest priority (smallest integer = highest priority).

SJF is a special case of priority scheduling where process priority is the inverse of the remaining CPU time. Equal-priority processes are scheduled in FCFS order.

Priorities can be defined:

- **Internally**: Use measurable quantities like time limits, memory requirements, the number of open files, and the ratio of average I/O burst to average CPU burst.
- **Externally**: Set by criteria external to the OS, such as the importance of the process, the type and amount of funds being paid for computer use, and the department sponsoring the work.

Priority scheduling can be preemptive or non-preemptive. When a process arrives at the ready queue, its priority is compared with the priority of the currently running process.

Problem: Starvation/Indefinite Blocking

Low-priority processes may never execute, leaving some processes waiting indefinitely for the CPU.

Solution: Aging

As time progresses, increase the priority of processes that wait in the system for a long time. For example, if priorities range from 127 (low) to 0 (high), decrement the priority of a waiting process by 1 every 15 minutes.

Example of Non-Preemptive Priority Scheduling

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Priority scheduling Gantt Chart:

```
P2   P5   P1   P3   P4
0   1   6   16   18   19
```

Waiting Time: P1 = 6; P2 = 0; P3 = 16; P4 = 18; P5 = 1. Average waiting time = (6+0+16+18+1)/5 = 41/5 = 8.2 msec

Multilevel Queue

> Processes are classified into different groups. A common division is foreground (interactive) and background (batch).

Why?

These processes have different response time requirements, so they need different scheduling. Foreground processes have higher priority over background processes.

How?

Multi-Level Queue Scheduling Algorithm (MQSA) partitions the ready queue into separate queues. Processes are permanently assigned to a given queue based on some property, such as:

- Process priority
- Process type
- Memory size

Each queue has its own scheduling algorithm:

- Foreground: RR
- Background: FCFS

Scheduling must be done between the queues:

- Fixed priority scheduling
- Time Slicing

Fixed Priority Scheduling

The foreground queue has absolute priority over the background queue. Serve all from the foreground, then from the background. A possibility of starvation exists.

Time Slice

Each queue gets a certain amount of CPU time, which it can schedule amongst its processes. For example, 80% of CPU time to the foreground in RR and 20% to the background in FCFS.

Advantages

1. Apply different scheduling methods to distinct processes.
2. Low scheduling overhead.

Disadvantages

1. Risk of starvation for lower-priority processes.
2. Rigid in nature.

Multilevel Feedback Queue

> A process can move between the various queues. The idea is to separate processes with different CPU burst characteristics.

If a process uses too much CPU time, it will be moved to a lower-priority queue. This scheme leaves I/O-bound and interactive processes in the higher-priority queues.

Introduction to Multilevel Feedback Queue

The **Multilevel Feedback Queue** is a type of scheduling algorithm that allows processes to move between different queues based on their priority and CPU burst time. This algorithm is designed to prevent **starvation**, which occurs when a process is unable to gain access to the CPU for an extended period of time.

Multilevel Feedback Queue Scheduling

The **Multilevel Feedback Queue** scheduling algorithm is defined by the following parameters:

- Number of queues
- Scheduling algorithms for each queue
- Method used to determine when to upgrade a process
- Method used to determine when to demote a process
- Method used to determine which queue a process will enter when that process needs service

Example of Multilevel Feedback Queue

Consider a system with three queues:

- Queue 0: RR with time quantum 8 milliseconds
- Queue 1: RR with time quantum 16 milliseconds
- Queue 2: FCFS

The scheduler first executes all processes in Queue 0. Only when Queue 0 is empty will it execute processes in Queue 1. Queue 2 will be executed only when Queue 0 and Queue 1 are empty.

Process Movement Between Queues

A process that arrives for Queue 1 will preempt a process in Queue 2. A process that arrives for Queue 0 will preempt a process in Queue 1.

Scheduling Algorithm

The scheduling algorithm works as follows:

- A new job enters Queue 0, which is served RR. When it gains CPU, the job receives 8 milliseconds.
- If it does not finish in 8 milliseconds, the job is moved to the tail of Queue 1.
- If Queue 0 is empty, the process at the head of Queue 1 is given a time quantum of 16 milliseconds.
- If it still does not complete, it is preempted and moved to Queue 2.

Advantages and Disadvantages

The advantages of Multilevel Feedback Queue scheduling are:

- It is more flexible
- It allows different processes to move between different queues
- It prevents starvation by moving a process that waits too long for the lower priority queue to the higher priority queue

The disadvantages of Multilevel Feedback Queue scheduling are:

- It produces more CPU overheads
- It is the most complex algorithm

Thread Scheduling

Thread scheduling is the process of allocating CPU time to threads. There are two types of thread scheduling:

- Process-Contention Scope (PCS): scheduling competition is within the process
- System-Contention Scope (SCS): scheduling competition is among all kernel threads in the system

Pthread

Pthread is a POSIX standard for threads. It provides a way to create and manage threads in a program.

> A thread is a separate flow of execution in a program. It is a lightweight process that can run concurrently with other threads.

The Pthread API provides functions for creating and managing threads, including:

- `pthread_create`: creates a new thread
- `pthread_join`: waits for a thread to finish
- `pthread_exit`: terminates a thread

Pthread Functions

The following table summarizes the Pthread functions:

Function	Description
`pthread_create`	creates a new thread
`pthread_join`	waits for a thread to finish
`pthread_exit`	terminates a thread
`pthread_attr_init`	initializes a thread attribute object
`pthread_attr_setscope`	sets the thread contention scope attribute
`pthread_attr_getscope`	gets the thread contention scope attribute

Linux Scheduling

Linux scheduling is the process of allocating CPU time to processes. The Linux scheduler is a preemptive priority-based algorithm.

> A preemptive scheduler is a scheduler that can interrupt a process and allocate the CPU to another process.

The Linux scheduler has two separate priority ranges:

- Real-time range: 0 to 99
- Nice value range: 100 to 140

Linux Scheduling Algorithm

The Linux scheduling algorithm works as follows:

- The scheduler chooses the task with the highest priority from the active array for execution on the CPU.
- When all tasks have exhausted their time slices, the active array and expired array are exchanged.

Completely Fair Scheduler (CFS)

The Completely Fair Scheduler (CFS) is a scheduling algorithm used in Linux. It is designed to provide fair scheduling and prevent starvation.

> Fair scheduling means that each process gets a fair share of CPU time.

The CFS algorithm uses a red-black tree to schedule processes. Each process has a virtual time associated with it, which is increased by the time it spends executing.

CFS Performance

The CFS algorithm provides good performance and fairness. It is able to handle a large number of processes and provide fair scheduling.

The following table summarizes the advantages of CFS:

Advantage	Description
Fair scheduling	each process gets a fair share of CPU time
Prevents starvation	prevents processes from being unable to gain access to the CPU
Good performance	able to handle a large number of processes

window.MathJax = {

```
tex: {
  inlineMath: [['$', '$'], ['\\(', '\\)']],
  displayMath: [['$$', '$$'], ['\\[', '\\]']]
}
```

```
};
```